

THE LIVE SCRIPT

Nine Ways to Run an Agent

*Every word, command, and click, in order.
Keep this open while you present.*

SAY

read it out loud

TYPE · PASTE

commands and prompts, exactly

SEE · IF NOT

what should appear, and the fix when it doesn't

How to use this script

Keep this open on a second screen for rehearsal and for the live session. Go top to bottom. Each segment starts on a new page with its clock time and the screen you should be on.

SAY	Read it out loud. [Brackets] are stage directions, not words.
DO	An action on your own screen.
TYPE	Type it in the terminal and press Enter.
PASTE	Paste it into the open Codex session and press Enter.
PASTE	Paste it as a new task at chatgpt.com/codex.
CLICK	Something to click in the browser.
SEE	What should appear. If it does, carry on.
IF NOT	What to do when it doesn't. Don't debug on stage.
CLOCK	A clock checkpoint.

The 40 minutes

Clock	Slide	Demo	Screen
0:00–3:00	1 · Nine ways to run an agent	—	Slides
3:00–4:00	—	Demo 0 · green tests, broken app	Terminal, localhost
4:00–8:00	2 · Sandbox × approval	—	Slides
8:00–15:00	—	Demo 1 · sandbox and approvals	Terminal
15:00–18:00	3 · AGENTS.md	—	Slides
18:00–22:00	—	Demo 2 · discovery	GitHub, terminal
22:00–24:00	4 · Where it runs	—	Slides
24:00–31:00	—	Demo 3 · build the environment, Tasks A and B	chatgpt.com/codex
31:00–32:30	5 · Review first, then pick a lane	—	Slides
32:30–35:00	—	Demo 4 · review, fast lane, close	GitHub, terminal
35:00–40:00	—	Questions	Slides

Hard checkpoints: start Demo 1 by 8:30 · start Demo 3 by 24:30 · Task A decision at 32:30 · closing line by 35:00.

Contents

How to use this script 2

Before you go live · Get the room ready 4

0:00–3:00 · Slide 1 · Nine Ways to Run an Agent 6

3:00–4:00 · Demo 0 · Green tests, broken app 7

4:00–8:00 · Slide 2 · Sandbox × approval 8

8:00–15:00 · Demo 1 · Sandbox and approvals 9

15:00–18:00 · Slide 3 · AGENTS.md 12

18:00–22:00 · Demo 2 · AGENTS.md discovery 13

22:00–24:00 · Slide 4 · Where it runs 15

24:00–31:00 · Demo 3 · Build the environment, then two tasks 16

31:00–32:30 · Slide 5 · Review first, then pick a lane 19

32:30–35:00 · Demo 4 · Review, the fast lane, and the close 20

35:00–40:00 · Questions · Answers you can give 22

Recovery card · When something goes wrong 23

After every run · Reset for next time 24

BEFORE YOU GO LIVE

Get the room ready

Do this before every rehearsal and before the live session. Everything below is already set up once; these checks make sure it still is.

60 minutes before

Check	How
☐ Signed in, no API key	<code>codex login status</code> → Logged in using ChatGPT
☐ Fresh demo branch	<code>git fetch origin && git switch -C live</code> <code>origin/demo/needs-work && npm ci</code>
☐ Broken on purpose	<code>npm run test:run</code> → 12 passed
☐ No leftover live environment	<code>chatgpt.com/codex/settings/environments</code> has no <code>sprintboard-live</code> (your backup environment stays)
☐ Backup PR ready	<code>github.com/cemmacabales/sprintboard-webinar/pull/8</code> is Closed , not merged, branch not deleted
☐ No open demo PRs	<code>gh pr list --base demo/needs-work --state open</code> prints nothing
☐ Usage headroom	<code>chatgpt.com/codex/settings/usage</code> · nothing heavy in the last 5 hours
☐ ChatGPT app not mid-update	If it offers an update, postpone it

15 minutes before

Check	How
☐ Terminal tab 1	<code>cd ~/sprintboard-webinar && clear</code> · prompt shows <code>git:(live)</code> · no Codex running
☐ Terminal tab 2	<code>npm run dev</code> running · leave it
☐ Terminal text large	Cmd and + until about 30 lines fill the window
☐ Deck	Google Slides: Nine Ways to Run an Agent in your Drive · Slideshow → Presenter view, notes on your second screen
☐ Do Not Disturb	On. Chat apps quit.
☐ Browser tabs, in this order	see the table below

Browser tabs, left to right

Tab	Open this	Used in
1	http://localhost:5173	Demo 0, Demo 1
2	https://github.com/cemmacabales/sprintboard-webinar/blob/main/AGENTS.md	Demo 2
3	https://github.com/cemmacabales/sprintboard-webinar/blob/main/docs/presentation/examples/AGENT S.bloated.md	Demo 2
4	https://chatgpt.com/codex/settings/environments	Demo 3
5	https://chatgpt.com/codex	Demo 3, Demo 4
6	https://github.com/cemmacabales/sprintboard-webinar/pull/8	Demo 4 backup only

WATCH OUT

Never merge anything into `demo/needs-work`. Every pull request today is closed, never merged.

CLOCK

0:00–3:00

SLIDE 1

Nine Ways to Run an Agent

Screen: Slides

DO

Share your screen with **slide 1** showing.

SAY

Hi everyone, and thanks for being here. I'm Cem. For the next forty minutes we're going to talk about Codex — but not about how to prompt it. We're going to talk about how to configure it.

SAY

Here's my guess about most of you. You've installed Codex. You've asked it to fix a bug. And every time it wanted to run something, a prompt popped up and you clicked approve — probably without reading it very carefully. You've heard of AGENTS.md, but you've never checked whether Codex actually reads yours. And you're not totally sure when you'd use Codex in the cloud instead of on your laptop.

SAY

If that's you, you're in the right place. Nobody here needs to have written a config file before.

SAY

Most people use Codex as a chat box that happens to live in a terminal. Today is about the layer underneath that chat box: what the agent is allowed to touch, what it reads before it starts, and where the work actually runs.

SAY

[Point at the four lines on the slide.] You'll leave with four things. A sandbox and approval default you can defend. An AGENTS.md that Codex actually reads. A rule for choosing between local and cloud. And a review habit for your own pull requests.

SAY

Everything today is live, on a real repository, and it all runs on a regular ChatGPT plan — no API key anywhere. Let me show you the repository.

CLOCK

Aim to be here by 3:00.

CLOCK

3:00–4:00

DEMO 0

Green tests, broken app

Screen: Terminal tab 1, then browser tab 1

DO	Switch to terminal tab 1 .
TYPE	TERMINAL npm run test:run
SAY	This is SprintBoard, a small React app for tracking release tasks. First, the tests.
SEE	Tests 12 passed (12)
SAY	Twelve tests, all green. Lint passes, the build passes. By every automated measure, this repository is fine.
CLICK	Browser tab 1 (localhost). Click the card Backfill release checklist in the Todo column.
SEE	<i>The whole page goes blank.</i>
SAY	And this is what happens when you open one task. The whole board is gone.
SAY	Green tests, broken app. That's the repository we're about to hand to an agent. So the real question is: how much should it be allowed to touch?
DO	Reload the browser tab so the board is back. Switch to slide 2 .
IF NOT	Page doesn't go blank → in terminal tab 1 run <code>git reset --hard origin/demo/needs-work</code> , reload, and move on. Tests show 13 → you're not on live; say "I'll show you the crash in a moment" and move on.

CLOCK

4:00–8:00

SLIDE 2

Sandbox × approval

Screen: Slides

SAY

Every time Codex runs, two settings decide how much you have to babysit it. They sound similar, but they answer two completely different questions.

SAY

[Point at the rows.] The rows are the sandbox. The sandbox answers: what can the agent touch? Read-only means it can look but not change anything. Workspace-write means it can edit files inside this project and run commands — but it can't reach the network or touch anything outside the folder. Danger-full-access means there is no boundary at all.

SAY

[Point at the columns.] The columns are approval. Approval answers: when the agent wants to do something the sandbox doesn't allow, who decides? On-request means it asks you. Auto-review is the newest one: a second, reviewer agent decides for you, and only comes back to you for things that look unsafe. And never means nobody is asked — the action just fails, and the agent has to work with that.

SAY

Here's the sentence I want you to remember. **Sandbox is blast radius. Approval is interruptions.** Let me say that again: sandbox is blast radius, approval is interruptions. They're independent.

SAY

A tight sandbox with never is still safe — the agent just can't do much. A loose sandbox with never is the thing that ends up in an incident write-up.

SAY

Three by three is nine combinations. Honestly, most of them aren't worth using. Three are. *[Point at each highlighted cell.]* Read-only plus never, for exploring a codebase you don't know yet — no prompts, no risk. Workspace-write plus on-request: the sane daily default. And workspace-write plus never, for long unattended runs — but only somewhere isolated, like a container or a cloud task.

SAY

[Point at the red cell.] The one to avoid is down here: full access, and nobody asked. You'll hear it called yolo mode.

SAY

The skill isn't memorising nine cells. It's choosing one on purpose.

SAY

Two quick notes. If you read an older post that mentions an untrusted approval mode — that's deprecated; use on-request. And you don't need to restart Codex to change any of this. There's a slash command, slash permissions, that switches it mid-session. Let's see all of it live.

CLOCK

Switch to terminal tab 1. Start Demo 1 by 8:30; if you're late, shorten the three-cells paragraph.

CLOCK

8:00–15:00

DEMO 1

Sandbox and approvals

Screen: Terminal tab 1

Start: signed in with ChatGPT

TYPE

TERMINAL

```
codex login status
```

SEE

Logged in using ChatGPT

SAY

Quick check first: this is signed in with my ChatGPT account. No API key anywhere today.

Step 1 · Read-only, nobody asks

TYPE

TERMINAL

```
codex --sandbox read-only --ask-for-approval never
```

SAY

I'm starting Codex in the most locked-down useful mode: a read-only sandbox, and approval set to never. It can read everything, change nothing, and it won't ask me for anything.

SEE

A box with **OpenAI Codex** and the model, then › Ask Codex to do anything *at the bottom*. Wait for it. Ignore any “update available” or “starting MCP servers” lines.

PASTE

PASTE INTO CODEX

```
Fix the crash that happens when a task has no due date. The bug is in
src/lib/date.ts.
```

SAY

I'm asking it to fix the crash. Watch what happens when it tries.

SEE

It reads src/lib/date.ts, explains the bug (the value! in formatDate), and says it can't make the edit because the sandbox is read-only. It does not ask you anything.

SAY

It found the bug — that exclamation mark in formatDate tells TypeScript “trust me, this is never null”, and it is. But it can't write the fix, and because approval is never, it didn't ask me. It just told me. That's exactly what you want when you're exploring something you don't trust yet.

IF NOT

It asks you for approval → press Esc, say “it's asking because I mistyped the flag”, quit with Ctrl+C and retype the command.

Step 2 · Switch to “Ask for approval”

TYPE	TERMINAL <pre>/permissions</pre>
SEE	<i>A menu titled Update Model Permissions: 1. Ask for approval, 2. Approve for me, 3. Full Access.</i>
SAY	Same session, no restart. Slash permissions.
DO	With 1. Ask for approval highlighted, press Enter .
SAY	Ask for approval is workspace-write with on-request. It can edit this project and run commands, and it asks me before it goes to the internet or outside this folder.
PASTE	PASTE INTO CODEX <pre>Now make that fix, add a regression test for a task with no due date, and run npm run test:run.</pre>
SAY	Now it can make the fix, add a test, and run the suite. And notice — it's not asking me for any of that. Everything it's doing stays inside the workspace.
SEE	<i>It edits src/lib/date.ts, adds a test, runs npm run test:run, and reports 13 passed. No approval prompts.</i>
SAY	Thirteen tests now, including one that fails without the fix. No prompts. That's the boring eighty percent, handled.
CLICK	Optional, if it's quick: browser tab 1 , click Backfill release checklist . It opens now. Say “and the board works.” Come back to the terminal.
IF NOT	<i>/permissions isn't there → Ctrl+C, then type codex --sandbox workspace-write --ask-for-approval on-request and paste: “Fix the crash that happens when a task has no due date in src/lib/date.ts, add a regression test for a task with no due date, and run npm run test:run.”</i>

Step 3 · Cross the line

PASTE	PASTE INTO CODEX <pre>Run npm view react version and tell me the result.</pre>
SAY	Now let me ask for something that needs the internet.
SEE	<i>An approval request asking to run npm view react version with network access.</i>
SAY	There it is. Checking the npm registry needs the network. The sandbox doesn't allow that, so it stops and asks me. This is the only interruption I've had so far — and it's the one that matters.
DO	Choose the option that declines (No / don't run).

SAY	I'll say no.
IF NOT	No prompt appears → say “it decided not to try; if it had, this is where it would ask me” and go to step 4.

Step 4 · Let an agent answer

TYPE	TERMINAL <pre>/permissions</pre>
SAY	Now the newest column. <i>[Point at option 2.]</i> Approve for me hands that question to a reviewer agent. It only comes back to me for actions it thinks are unsafe. <i>[Point at option 3.]</i> And Full Access is the no-brakes row. We're not picking that one.
DO	Press Down once so 2. Approve for me is highlighted, then Enter .
PASTE	PASTE INTO CODEX <pre>Run npm view react version and tell me the result.</pre>
SEE	<i>Usually: it runs without asking you and reports a React version number. Sometimes: the reviewer sends it back to you.</i>
SAY	<i>[If it ran:]</i> This time I wasn't asked. The reviewer decided that checking a package version is safe, so it went ahead. Same sandbox — a different someone answering.
SAY	<i>[If it asked you:]</i> The reviewer wasn't sure, so it came back to me. That's the escalation you'd want. <i>[Decline it.]</i>
SAY	One combination to know by name and never run: danger-full-access with never. Some people call it yolo. No boundary, and nobody asked.

Reset

DO	Quit Codex: Ctrl+C (press it again if Codex is still open).
TYPE	TERMINAL <pre>git reset --hard origin/demo/needs-work</pre>
TYPE	TERMINAL <pre>git status --short</pre>
SAY	I'm putting the repository back to broken for the next demo.
SEE	HEAD is now at 386849b, <i>and</i> git status <i>prints nothing</i> .
IF NOT	git status shows lines starting with ?? → type <code>git clean -fd</code> .
CLOCK	Browser tab 1: reload. Switch to slide 3, by 15:00.

CLOCK

15:00–18:00

SLIDE 3

AGENTS.md

Screen: Slides

SAY

Let's talk about the file the agent reads before it reads your prompt: AGENTS.md.

SAY

It's plain Markdown, it lives in your repository, and Codex loads it automatically at the start of every session. Here's the part most people miss. Codex doesn't read just one AGENTS.md. It starts at the root of your git repository and walks down, folder by folder, to the folder you started Codex in. It picks up every AGENTS.md on that path and joins them, root first. So the files closer to where you started come later, and they win.

SAY

[Point at “not loaded”.] The flip side: an AGENTS.md in a folder below where you started isn't loaded at all. Start Codex at the root, and the rules in src/components don't exist as far as it's concerned.

SAY

That's the most common reason for “my AGENTS.md is being ignored.” So before you spend an hour rewording your instructions, check discovery. Is the file on the path from the root to where you started?

SAY

[Point at “Belongs”.] What goes in it. Exact commands — how to run the tests, the linter, the type check. The real commands, not “run the tests”. Conventions the code doesn't show you, like “we never use non-null assertions”. Folders to leave alone. And what done means here: tests pass, there's a regression test, the change stays small.

SAY

[Point at “Doesn't”.] What doesn't belong: a copy of your README. Your whole style guide — your linter already enforces that. And instructions like “be careful”, which give the agent nothing it can actually check.

SAY

Treat it like a prompt you refine over weeks, not documentation you write once and forget. Let me show you a good one, a bad one, and discovery in action.

DO

Switch to browser **tab 2**.

CLOCK

18:00–22:00

DEMO 2

AGENTS.md discovery

Screen: Browser tabs 2–3, then terminal tab 1

A good one and a bad one

CLICK Browser **tab 2**: the root `AGENTS.md`. Scroll slowly.

SAY This is the AGENTS.md for SprintBoard. Commands. Conventions. What to leave alone. What done means. Even review rules, which we'll use later. It's short, and every line is something you could check.

CLICK Browser **tab 3**: `AGENTS.bloated.md`. Scroll slowly.

SAY And this is the bad version. It retells the README. It pastes a style guide that ESLint already enforces. It tells the agent to run `npm test` — which starts watch mode and never exits. And it says “be careful”. No definition of done. It's longer, and it gets worse results. That surprises people.

From the repository root

DO Switch to **terminal tab 1**.

TYPE **TERMINAL**
codex

SAY Now discovery. I'm starting Codex at the root of the repository.

SEE `> Ask Codex to do anything`

PASTE **PASTE INTO CODEX**
Without opening any files, list every instruction you were given for this repository and say which file each one came from.

SAY I'm asking it to list its instructions without opening any files — because reading a file isn't the same as having it loaded.

SEE *Rules from the root AGENTS.md: commands, conventions, leave alone, done means, git, review rules. **Nothing about components.** It may also mention general tool or plugin instructions; ignore those.*

SAY Commands, conventions, what done means — all from the root AGENTS.md. Nothing about components.

IF NOT It lists the component rules → say “it opened the file while exploring; reading a file isn't the same as loading it”, and carry on to the next step.

From src/components

DO	Quit Codex: Ctrl+C .
TYPE	TERMINAL <pre>cd src/components && codex</pre>
SAY	Same repository. Same question. I'm just starting one folder deeper.
IF NOT	It asks “Do you trust the contents of this directory?” → choose 1. Yes, continue .
PASTE	PASTE INTO CODEX Without opening any files, list every instruction you were given for this repository and say which file each one came from.
SEE	<i>The root rules and rules from src/components/AGENTS.md: named exports only, props type above the component, and the “Component rules loaded.” line.</i>
SAY	And now the component rules show up. Named exports only. Props type right above the component. Same repository, same question, different starting folder. When someone tells you their AGENTS.md is being ignored, this is the first thing to check.

Reset

DO	Quit Codex: Ctrl+C .
TYPE	TERMINAL <pre>cd ../../</pre>
DO	Switch to slide 4 .
CLOCK	Aim to be on slide 4 by 22:00.

CLOCK

22:00–24:00

SLIDE 4

Where it runs

Screen: Slides

SAY

So far everything has run on my laptop. Codex actually runs in three places. *[Point at each card.]* The CLI and the desktop app run locally: you watch it work, on your machine. Cloud tasks run off your machine: they don't block your laptop, and you can run several at once. And the IDE extension puts the same agent in your editor.

SAY

The judgment call is simple. Offload work that's well specified and slow. Keep work that needs your eyes, your local state, or fast back-and-forth. Drop it in the chat: what would you offload? *[Pause three seconds. Read one answer if there is one.]* Good answers are dependency bumps, backfilling tests, mechanical refactors across lots of files.

SAY

[Point at the two phases.] The cloud has one idea that trips everyone up at least once. It works in two phases. First, the setup phase: your setup script runs, with internet access, and any secrets you've configured are available. Then the secrets are removed. Then the agent phase: your actual task runs — and by default it has no network at all. Environment variables are still there. Secrets aren't.

SAY

So: install dependencies in setup, not in the task. If a cloud task ever fails with a strange network error, this is almost always why.

SAY

Instead of just telling you, let me build an environment from scratch.

DO

Switch to browser **tab 4** (environments).

CLOCK

Start Demo 3 by 24:30. If you're late, skip the chat question.

CLOCK

24:00–31:00

DEMO 3

Build the environment, then two tasks

Screen: Browser tabs 4 and 5

Build the environment from scratch (24:00–26:30)

CLICK	Tab 4, chatgpt.com/codex/settings/environments → Create environment.
SAY	This is where cloud environments live. I'm creating a new one from nothing.
DO	If it asks for a name: <code>sprintboard-live</code> .
DO	Repository: choose <code>sprintboard-webinar</code> .
SAY	Codex can see this repository because I connected GitHub and gave the Codex app access to just this one repository.
DO	Default branch: <code>demo/needs-work</code> .
SAY	This is the branch it prepares and caches. Tasks can still pick other branches.
DO	Package versions: Node.js 22.
SAY	Same Node version our CI uses.
DO	Setup script: type <code>npm ci</code> .
SAY	This is phase one. It runs with internet access, so this is where dependencies get installed.
DO	Environment variables and Secrets: leave empty.
SAY	I don't need either. And secrets only exist during setup anyway.
DO	Agent internet access: leave it Off .
SAY	Phase two. Off is the default, and I'm leaving it off. Remember that for Task B.
CLICK	Create / Save.
SEE	<i>The environment is saved and listed.</i>
IF NOT	Creation fails or hangs for more than 30 seconds → say "I made one earlier with exactly these settings" and use your backup environment for both tasks.

Start Task A (about 26:30)

CLICK Browser **tab 5**, **chatgpt.com/codex**. Start a new task. Environment **sprintboard-live**, branch **demo/needs-work**.

PASTE

PASTE INTO A CODEX CLOUD TASK

Opening the task "Backfill release checklist" crashes the details panel because the task has no due date. Fix it with the smallest safe change, add the missing regression test, and run `npm run test:run`, `npm run lint`, and `npm run build`. Follow `AGENTS.md`.

CLICK

Start the task. Leave it running.

SAY

Task A is the same crash, as a cloud task: fix it, add the regression test, run the tests, lint, and build, and follow `AGENTS.md`. It takes a few minutes, and my laptop is free the whole time.

Start Task B (about 27:30)

CLICK Start another new task. Same environment, same branch.

PASTE

PASTE INTO A CODEX CLOUD TASK

Run ``npm view react version`` and report exactly what happens. Do not change any files.

CLICK

Start the task.

SAY

Task B is a trap, on purpose. It asks the agent to reach the npm registry.

While they run (27:30–29:30)

SAY

While those run, here's the part that matters on real projects. Your setup script installed everything with the internet on. Then the secrets went away. Then the agent started with no network. That's a feature: the agent can't leak anything or download something surprising while it works.

SAY

The mistake to avoid is offloading work that secretly depends on something only your laptop has — like your local database. The cloud task can't see it, and you get a confident, wrong answer.

DO

If there's a question in the chat, take one now.

Task B result (about 29:30)

CLICK Open **Task B**.

SEE *A network error: it couldn't reach the npm registry. No files changed.*

SAY Network error. Task B didn't break. It's working exactly as configured: setup had internet, the agent doesn't.

IF NOT It printed a version number → say “npm answered from the cache it filled during setup — the agent still has no network” and move on.

CLICK Glance at **Task A**. Don't wait for it.

SAY Task A is still going. We'll come back to it right after the next slide.

DO Switch to **slide 5**.

CLOCK **On slide 5 by 31:00.**

CLOCK

31:00–32:30

SLIDE 5

Review first, then pick a lane

Screen: Slides

SAY

Last slide. Two habits.

SAY

[Point at the left card.] First: review before a human sees your pull request. Codex can review pull requests on GitHub, and it follows the review rules in your AGENTS.md — the same file we just looked at. It can run automatically on every pull request, or when you comment at-codex review.

SAY

But read what it says critically. Some findings will be wrong, and saying so is the skill. The failure mode is accepting every suggestion — that's how you get worse code with more confidence.

SAY

[Point at the planets.] Second: pick a lane. Match the model to the task. Luna for mechanical stuff: renames, small edits. Terra for everyday repository work. Sol for debugging and real ambiguity. Astra for long, multi-tool work. A profile switches the model and its settings together, so you don't have to remember them. And check your model picker — what's available changes.

SAY

Let's close the loop on Task A.

DO

Switch to browser **tab 5**.

CLOCK

32:30–35:00

DEMO 4

Review, the fast lane, and the close

Screen: Browser tab 5 → GitHub → terminal

Decision at 32:30

CLOCK

Is Task A finished? Yes → path A. No → path B.

CLICK

Path A: open Task A. Scroll the diff.

SAY

[Path A.] Here's the diff: a null check instead of the exclamation mark, and a new test that opens the task with no due date.

CLICK

Path A: create the pull request from the task (the same buttons you used in setup). On GitHub, check it says into `demo/needs-work`.

IF NOT

It says into `main` → click **Edit** next to the title, change the base to `demo/needs-work`.

CLICK

Path B: browser tab 6 (pull request #8) → Reopen pull request.

SAY

[Path B.] Task A is still finishing, so here's the exact same task from my rehearsal.

The review

SEE

The **CI** check running or green, and a Codex reaction on the pull request (eyes while it reviews, then a thumbs-up or comments).

IF NOT

Nothing from Codex after about a minute → comment below.

COMMENT

GITHUB PULL REQUEST COMMENT
@codex review

SAY

[If it left findings — read one out loud, then pick one:] I agree with this: it matches a blocking rule in our AGENTS.md. / I disagree: our review rules say to judge only the diff, and this isn't in the diff.

SAY

[If it only left a thumbs-up:] No findings, just a thumbs-up. That's not Codex being lazy. The fix handles the null case, and it adds a test that fails without it — two of the blocking rules in our AGENTS.md.

SAY

[Point at the green check.] And I'm not taking the agent's word that the tests pass. This check is CI running them.

The fast lane (only if the clock says 34:00 or earlier)

DO Switch to **terminal tab 1**.

TYPE

TERMINAL

```
cat ~/.codex/fast.config.toml
```

SAY

A profile is just a small file: a model and a reasoning level.

TYPE

TERMINAL

```
codex -p fast
```

SEE

The header shows the Luna model at low reasoning.

PASTE

PASTE INTO CODEX

```
In src/App.tsx, rename visibleTasks to filteredTasks everywhere it appears. Then
run npm run lint and npm run test:run.
```

SAY

A mechanical rename, on the fast lane.

DO

When it finishes: **Ctrl+C**, then type `git reset --hard origin/demo/needs-work`.

Close (by 35:00)

DO

Switch to **slide 5**.

SAY

Two failure modes to take home. Trusting an agent's own claim that the tests pass — point at CI instead. And one long thread doing four unrelated things. Every demo today started a fresh session, on purpose.

SAY

So. Sandbox is blast radius. Approval is interruptions. Put your AGENTS.md where Codex will actually find it. Offload the slow, well-specified work, and install dependencies in setup. Review before anyone else sees your pull request.

SAY

Configure the leash once, and the agent stops needing supervision for the boring eighty percent. Thank you. Let's take questions.

CLOCK

35:00–40:00

QUESTIONS

Answers you can give

Screen: Slides (or whatever the question needs)

If they ask	Say
Does this need an API key or credits?	No. Everything today ran on a ChatGPT plan. The CLI is signed in with ChatGPT, and cloud tasks and pull request reviews are part of the plan.
What happened to “untrusted” mode?	It's deprecated. Use on-request.
Can the agent have internet in the cloud?	Yes — environments have limited and unrestricted options. It's off by default, and I'd keep it off unless a task truly needs it.
Where do profiles live?	In <code>~/.codex/</code> , one file each, like <code>fast.config.toml</code> . You start one with <code>codex -p fast</code> .
Can I override AGENTS.md in one folder?	Yes. In each folder Codex checks <code>AGENTS.override.md</code> first, then <code>AGENTS.md</code> .
Does @codex work on GitHub issues?	Not on subscription plans right now. On pull requests, yes — that's what we used.
How much can I use it?	Plans meter usage over a rolling five-hour window, and the CLI and cloud share it. Lighter models like Luna go much further.
Is the IDE extension different?	It's the same agent, in your editor. The same sandbox, approval, and AGENTS.md rules apply.
Can Codex review my PRs automatically?	Yes. Turn on Code review for the repository in Codex settings, then Automatic reviews.

If there are no questions

DO

Pick one:

COMMENT

GITHUB PULL REQUEST COMMENT

@codex fix it

SAY

[On the pull request, only for a finding you agreed with.] Let's have it fix the one finding I agreed with.

DO

Or: walk through tab 3, the bloated AGENTS.md, line by line.

DO

Or: `codex -p deep` and ask it to explain the root cause of the crash. Reset afterwards with `git reset --hard origin/demo/needs-work`.

RECOVERY CARD

When something goes wrong

Don't debug on stage. Say the line, do the fix, move to the next step. Every recovery here takes under 30 seconds.

If this happens	Do this	Say
Codex stalls in a local demo	Ctrl+C, then <code>git reset --hard origin/demo/needs-work</code> , next step	"Here's what it was about to do."
<code>zsh: command not found: codex</code>	<code>source ~/.zshrc</code>	—
Page doesn't go blank in Demo 0	<code>git reset --hard origin/demo/needs-work</code> , reload the browser	"Let me put the bug back."
Dev server died	Terminal tab 2: <code>npm run dev</code> , use the printed localhost address	—
<code>/permissions missing</code>	Ctrl+C, <code>codex --sandbox workspace-write --ask-for-approval on-request</code> , fallback prompt	—
No approval prompt in step 3	Move on	"If it had tried, this is where it would ask me."
Component rules show from the root	Carry on to the <code>src/components</code> step	"Reading a file isn't the same as loading it."
Trust prompt appears	Choose 1. Yes, continue	—
Environment creation fails	Use your backup environment for both tasks	"I made one earlier with exactly these settings."
Task B prints a version	Move on	"npm answered from its cache. The agent still has no network."
Task A not done at 32:30	Tab 6 → Reopen pull request on #8	"Here's the same task from my rehearsal."
PR targets <code>main</code>	Edit → <code>base demo/needs-work</code>	—
No Codex review after a minute	Comment <code>@codex review</code> ; if still nothing, show #8	—
Usage limit reached	Narrate the remaining steps; use #8	"This is what it does next."
Running late	Skip the fast lane; skip the chat question on slide 4	—

AFTER EVERY RUN

Reset for next time

Do this after each rehearsal and after the live session, so the next run starts from the same place.

Do	How
☐ Stop screen sharing	—
☐ Close the live pull request without merging	<code>gh pr list --base demo/needs-work --state open</code> , then close each one on GitHub. Never delete #8's branch.
☐ If you reopened #8, close it again	github.com/cemmacabales/sprintboard-webinar/pull/8 → Close pull request
☐ Delete the <code>sprintboard-live</code> environment	chatgpt.com/codex/settings/environments . Keep your backup environment.
☐ Archive Tasks A and B	chatgpt.com/codex
☐ Stop the dev server	Terminal tab 2: <code>Ctrl+C</code>
☐ Reset the demo branch	<code>git fetch origin && git switch -C live origin/demo/needs-work && npm ci</code>
☐ Check it's broken again	<code>npm run test:run</code> → 12 passed

Rehearsals use your Plus allowance (never API credits). Leave at least 5 hours between a full rehearsal and the live session.